

Los Evita Bloqueos

Consultoría de optimización de software, nuestro objetivo es ser consultoría para comparar soluciones secuenciales y paralelas, medir cuándo OpenMP produce una mejora real y documentar los casos donde el overhead o el hardware limitan el escalamiento. Somos una consultoría de software que busca mejorar el rendimiento de programas mediante la paralelización y optimización de código.

Integrantes

- **Vianka Vanessa Castro Ordoñez** — 23201
- **Mia Alejandra Fuentes Mérida** — 23775
- **Jorge Luis Felipe Aguilar Portillo** — 23195

Repositorio

- GitHub: [FelipeAP04/ParcialParalelas_1](https://github.com/FelipeAP04/ParcialParalelas_1)
- URL para clonar: https://github.com/FelipeAP04/ParcialParalelas_1.git

```
git clone https://github.com/FelipeAP04/ParcialParalelas_1.git
cd ParcialParalelas_1
```

Problemas elegidos

1. **Multiplicación de matrices densas:** versión secuencial con recorrido `i-k-j` y versión OpenMP por bloques de `32 x 32` usando `collapse(2)` y `schedule(static)`.
2. **Filtro blur en escala de grises:** versión secuencial con ventana deslizante y versión OpenMP dividida por filas usando `schedule(static)`.

Las entradas se generan de forma determinista. Cada programa imprime un checksum que permite comprobar que la versión secuencial y la paralela producen el mismo resultado.

Estructura

```
/secuencial
  matrices_secuencial.c
  blur_secuencial.c
/paralelo
  matrices_paralelo.c
  blur_paralelo.c
/docs
  contexto_y_datos.md
  estrategia_paralelizacion.md
/resultados
  /Mia_Fuentes
  /Vianka_Castro
```

```
benchmark.ps1
generar_graficas.ps1
```

Cada carpeta individual de resultados contiene el CSV de corridas, estadísticas resumidas, análisis de speedup/eficiencia y gráficas.

Compilación

Se necesita GCC con soporte para OpenMP. Desde la raíz del proyecto:

```
gcc -O2 -fopenmp secuencial/matrices_secuencial.c -o matrices_secuencial
gcc -O2 -fopenmp paralelo/matrices_paralelo.c -o matrices_paralelo
gcc -O2 -fopenmp secuencial/blur_secuencial.c -o blur_secuencial
gcc -O2 -fopenmp paralelo/blur_paralelo.c -o blur_paralelo
```

En PowerShell con MSYS2 instalado en `C:\msys64`:

```
$env:PATH = "C:\msys64\ucrt64\bin;$env:PATH"
gcc -O2 -fopenmp secuencial/matrices_secuencial.c -o matrices_secuencial.exe
gcc -O2 -fopenmp paralelo/matrices_paralelo.c -o matrices_paralelo.exe
gcc -O2 -fopenmp secuencial/blur_secuencial.c -o blur_secuencial.exe
gcc -O2 -fopenmp paralelo/blur_paralelo.c -o blur_paralelo.exe
```

Pruebas rápidas de correctitud

```
.\matrices_secuencial.exe 512
.\matrices_paralelo.exe 512 4 32

.\blur_secuencial.exe 1920 1080
.\blur_paralelo.exe 1920 1080 4
```

Para cada problema, ambos checksums deben ser exactamente iguales.

Benchmark individual

El script hace una corrida de calentamiento y 5 corridas medidas para cada configuración. Evalúa matrices de 1024, 1536 y 2048; blur en 3840x2160 y 7680x4320; y OpenMP con 1, 2, 4 y 8 hilos.

```
.\benchmark.ps1 -Integrante "Nombre Apellido"
```

Para una validación corta antes del benchmark completo:

```
.\benchmark.ps1 -Integrante "Nombre Apellido" -Quick
```

Los archivos se crean en `docs/resultados/Nombre_Apellido/`. Para generar las cuatro gráficas desde el CSV resumen:

```
.\generar_graficas.ps1 -Integrante "Nombre_Apellido"
```

Resultados disponibles

- [Resultados de Mia Fuentes](#)
- [Resultados de Vianka Castro](#)

Métricas

```
Speedup(p) = tiempo_secuencial / tiempo_paralelo(p)
Eficiencia(p) = Speedup(p) / p
Eficiencia (%) = Eficiencia(p) x 100
```

No se debe concluir que más hilos siempre son mejores. Los resultados se interpretan considerando overhead, caché, ancho de banda de memoria, núcleos físicos, hilos lógicos y variación entre corridas.